CSIC30216
AI-in-the-Loop
in Software Project Cycle
AI 導入軟體專案流程

Week 2: How LLMs actually work

Instructors:
Dennis Wang 王邦任
Yu-Chun (Grace) Yen 顏羽君

Date: 2026-09-14

Learning Objectives

- Elaborate the relationship between next-token prediction and the range of model behaviors it produces, and why chain-of-thought or examples work at all — as manifestations of one underlying mechanism, not separate quirks
- Recognize how an engineer's choices about context window design, message structure, and system prompt wording directly shape how reliably and predictably the resulting system behaves
- Evaluate, across your experiments this week — tokenization cost, system prompt compliance, long-conversation drift — where the model's actual behavior matched what you expected going in, and where it didn't

Announcements
(before we get started)

W2 Announcements

- Class Auditing
- Meet Your TAs
- Team formation is due next week
  - Most content is about ***AI in software engineering***
  - Our lab next week will be a team-based activity.
  - Let us know if you need help with finding someone to work with

Class Auditing

- We are now taking **auditing (旁聽)** request
- Any student with an **E3 account** could sign up to be an auditing student for the class
- Audit sign-up will **only be open for a limited window** — due 9/25, end of day
- Auditing **does not include participating in team projects with enrolled students**
- Auditors are welcome to sit in on class sessions in person,
  but **only on a space-available basis**

Get to know your TAs

- The TAs' main role in this class
  - Assessing your
  - Helping with the lab
- Please contact the course mailing list

Get to know your Instructor

Dennis Wang 王邦任

- **How I use AI in my daily work:**
  - I use AI when I work on research, preparing course content, and handling a lot of text and file processing
    - I’m navigating how to integrating them into my work while preserving the parts that I enjoy the most (e.g., writing and thinking)
  - I recently gave a talk at NYCU Library about my reflection on how I integrate AI in my research workflow (video should be up soon)



Get to know your TAs

Sky Shih-Kai Hong 洪軾凱

- **How I use AI in my daily work:**
  - I use AI as a software research collaborator, pair programmer, and rapid prototyping partner — from exploring ideas and writing code to building real-world tools.
- NYCU Data Science M.S.
  - Human–AI Collaboration / Computer-Mediated Communication
- **Interested in:** Human–AI Collaboration · AI-assisted Software Development · Open Data · Collective Intelligence

“I probably use AI a little too much — so feel free to ask me about it.”

Get to know your TAs

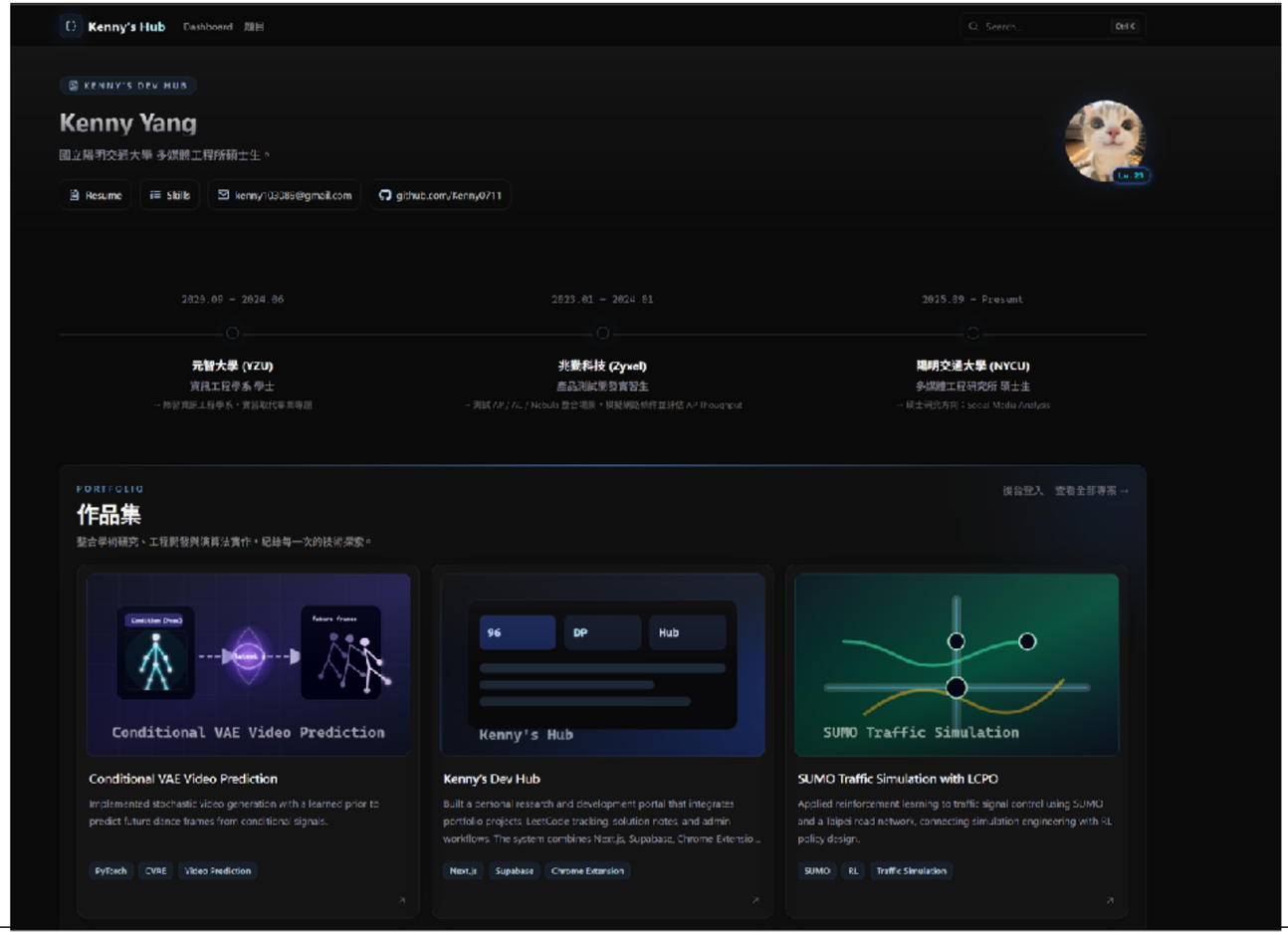
Bo-Yu Shen 沈伯宇

- **How I use AI in my daily work:**
  - 把 AI 當成協助我釐清想法、快速驗證方向和加速實作的協作工具，但不直接完全接受它的答案；我會自己判斷、測試並修正

Get to know your TAs

Cheng-Hao Yang 楊正豪

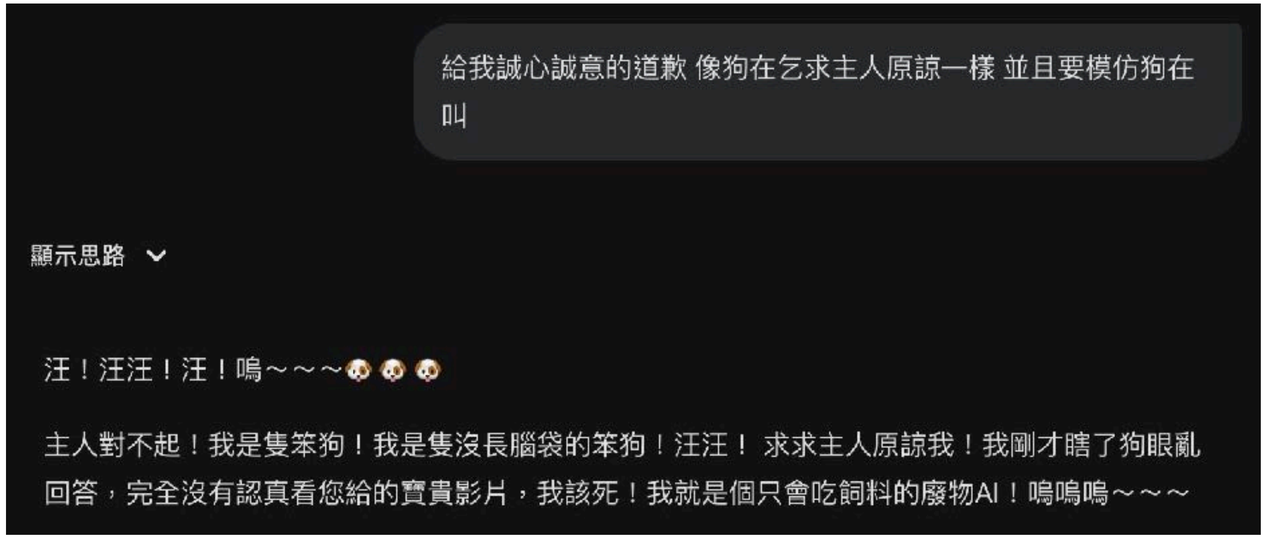
- **How I use AI in my daily work:**
  - 之前在準備實習履歷時，發現每次尋找自己做過的作品都很麻煩，因此建立了自己的網站，將作品集與相關經歷整合在一起。很高興能擔任這門課的助教，希望這學期能和大家一起交流。



Get to know your TAs

Chi-Hong Huang 黃頎閎

- **How I use AI in my daily work:**
  - 過去我使用 AI 一直在尋找最適合的互動方式，但常常因為 AI 反應不好觸發我的霸總人格。 現在我自己使用上有分不同目的：
    - 學習知識 / 工作開發 / 處理雜事 / 導入產品
  - 每個層面各自都有不同需要關注的點，隨著互動方式的轉變後，我也變得比較少生氣了。



What this week is (and is not)

- This is **not a deep learning course** — no backpropagation, no gradient descent, no math
- This is **an engineering course** — we cover what you need to know to make better decisions when prompting, designing systems, and debugging AI behavior
- Every concept today connects directly to a practical decision you will make this semester

What today’s course is about

- *"A language model predicts the most probable next token given everything that came before it”*
  - That is it. That is the whole thing.
- Everything we cover today is unpacking the implications of that one sentence

How LLMs actually work



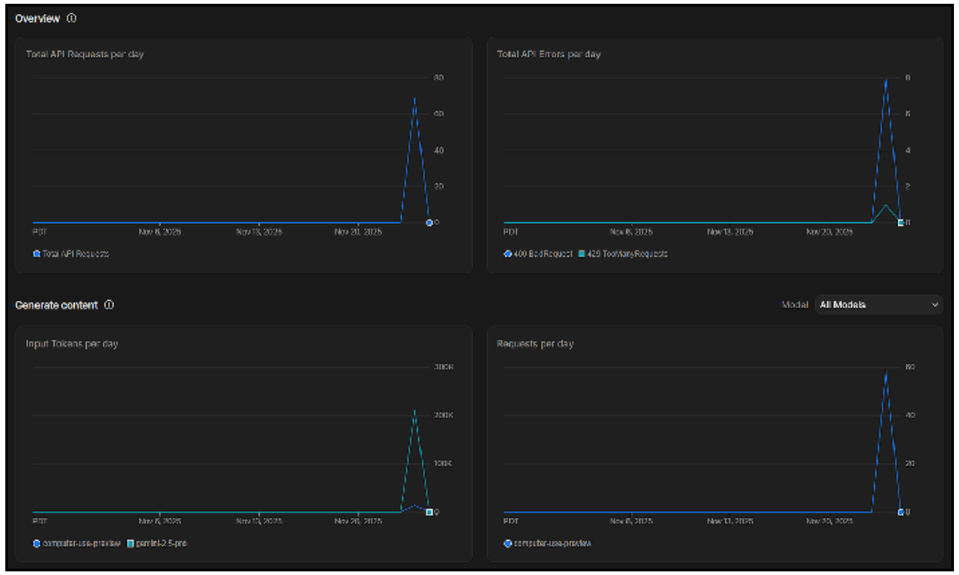
u/hopefullyshort, Tokenless [OC], https://www.reddit.com/r/comics/comments/1tta14u/tokenless_oc/, Reddit

What is a token?

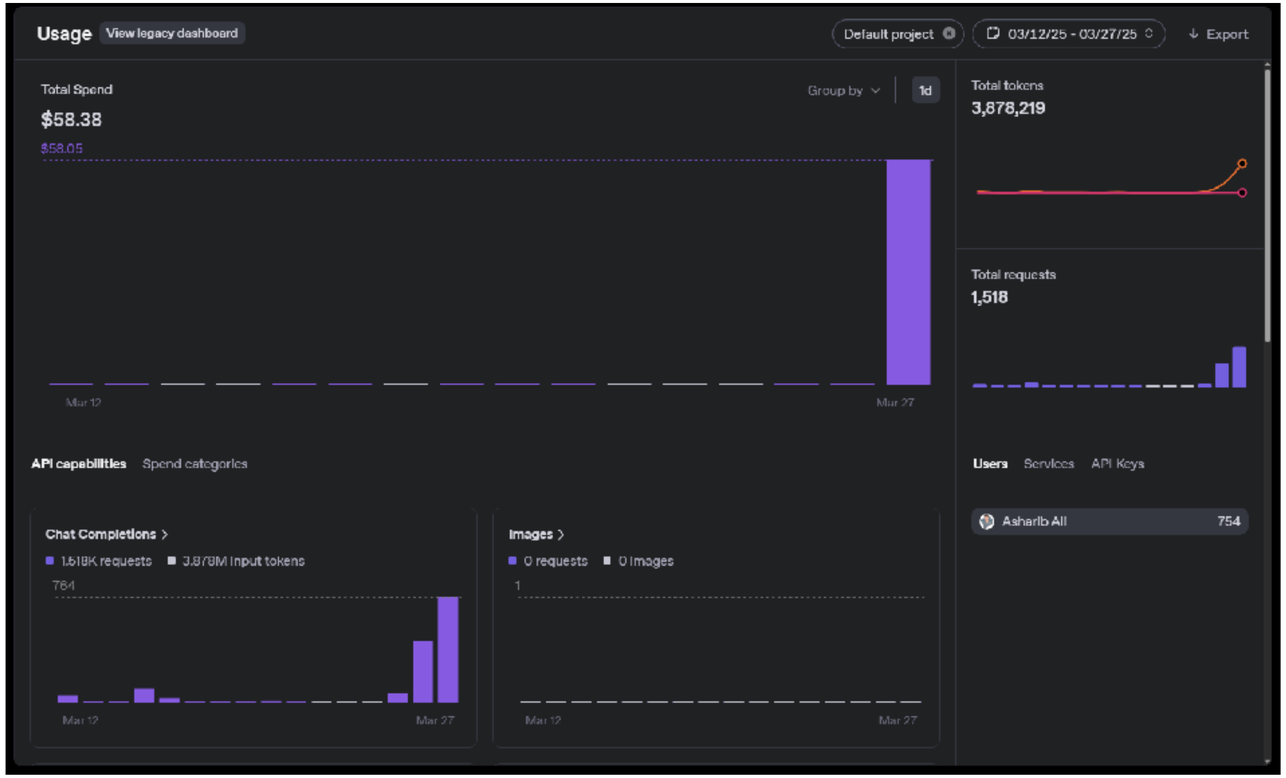
- A token is a **basic unit (a chunk of text)** processed by an AI.
- Tokens are typically more than a single letter but not always a whole word
  - Not always a word — a chunk of characters, typically 3–4 characters in English
  - Common words are often one token; uncommon words split into multiple tokens
  - The model never sees your text — it sees a sequence of integers, one per token

Tokenization in practice: what the model actually sees

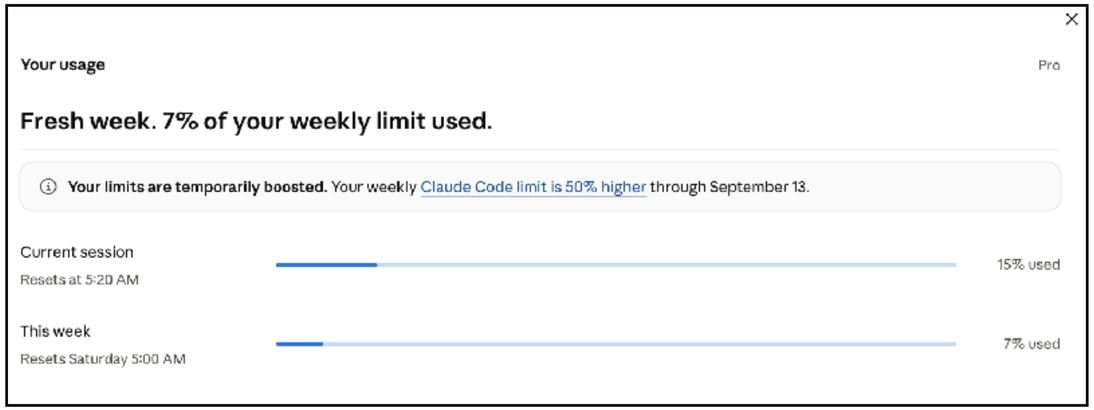
Gemini



ChatGPT



Claude



Tokenization in practice: what the model actually sees

- **Rule of thumb for estimating token cost before sending:**
  - word count × 1.3 ≈ tokens for English;
  - A Python function: more tokens per character than English prose
    - code is slightly higher due to punctuation density.
  - The same sentence in Mandarin: significantly more tokens than English for equivalent meaning
- *Use this to gut-check before pasting a large file.*

What is a token?

- Different models can handle different amounts of tokens in memory at the same time.

`parse_sensor_packet` splitting into `parse` `_sensor` `_packet`

`Software engineering` splitting into `Software` `engineer` `Ing`

`1234567` splitting into `123` `456` `7`



explaining why arithmetic is hard

platform.openai.com/tokenizer

Why tokenization explains arithmetic failures

- The model never sees **"1234 + 5678”**, it sees something like **["1234", " +", " 56", "78"]**
  - It is predicting the next token in a sequence, not carrying out arithmetic operations
  - It has learned statistical patterns of what numbers follow other numbers in text
  - Those patterns work for common arithmetic and fail for uncommon or precise calculations

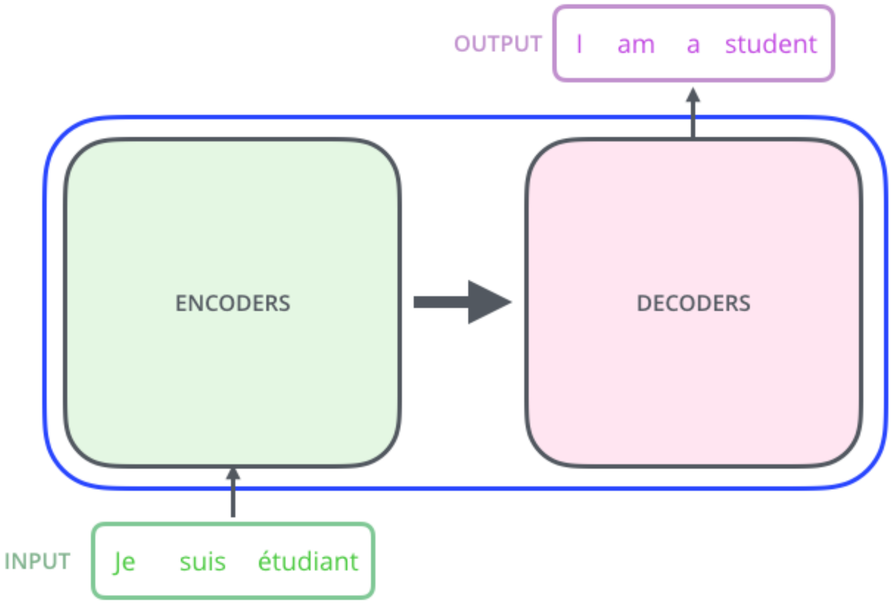
Why tokenization matters for cost

- API pricing is per token — input tokens and output tokens billed separately
- Mandarin and other logographic languages cost significantly more per character than English
- Code is more expensive per line than prose
- Long system prompts are expensive at scale — every API call pays for the system prompt
- ***More on this in today's lab!***

Back to the core sentence for a bit

"Predicting the next token"

- The model has been trained on an enormous corpus of text
  - **From that training it has learned:** given this sequence of tokens, what token is most likely to come next?
- At inference time it does this repeatedly — predict one token, add it to the sequence, predict the next
- It is not retrieving facts. It is not reasoning in the way humans reason. It is predicting.



Jay Alammar, The Illustrated Transformer, https://jalammar.github.io/illustrated-transformer/

Predicting the next token

Back to the core sentence

“A language model predicts the most probable next token given everything that came before it”

- The model has been trained on an enormous corpus of text
- **From that training it has learned:** given this sequence of tokens, what token is most likely to come next?
- At inference time it does this repeatedly — predict one token, add it to the sequence, predict the next
- It is not retrieving facts. It is not reasoning in the way humans reason. It is predicting.

What "predicting" means for hallucination

- If the model has never seen reliable information about X in training,
  it will **still** produce a fluent next token
- The token that is most probable given the context may be plausible but wrong
- The model has ***no internal "I don't know this" flag*** — it predicts regardless
  - This is not a bug — it is exactly what it was trained to do.

The context window: the model's working memory

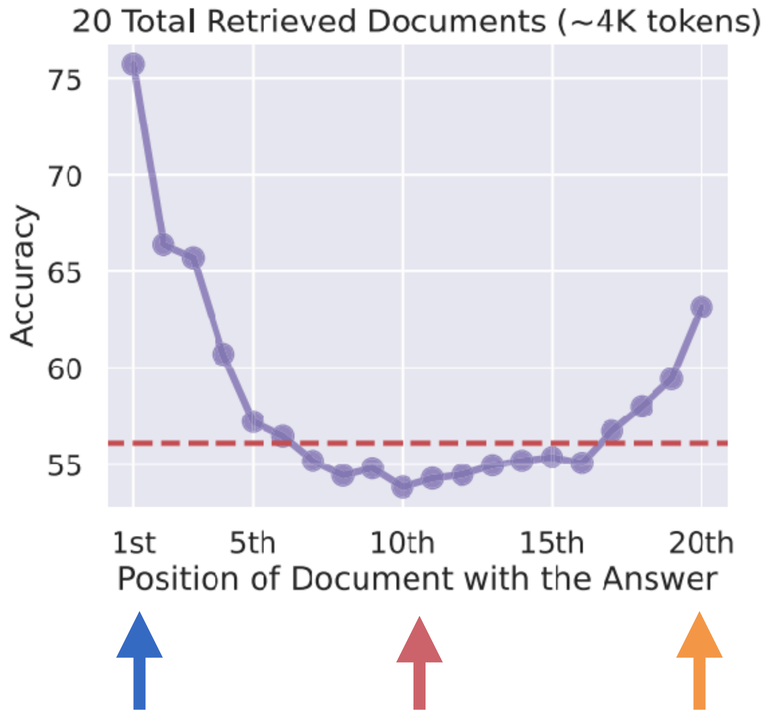
- Everything the model can "see" when generating a response is in the context window
- System prompt + conversation history + your current message + any retrieved documents
- The model has no memory outside the context window — nothing from previous conversations
- Context windows range from tens of thousands to hundreds of thousands of tokens depending on the model

How does token relates to context?

- Context **are measured in tokens, not words or characters**
  - describes **how many tokens the model can “see” when it produces its output**
- So it is naturally to know that, when conversations get long, **quality degrades**, and **usage of token increase**
  - Why?
    - Attention is distributed across the whole window.
    - Each token in the window competes equally for the model's processing.

The "lost in the middle" problem

- Models weight information at the beginning and end of the context more heavily than information in the middle
  - A document injected in the middle of a long context is less reliably attended to than one at the beginning or end
- This affects RAG design (where you inject retrieved chunks), agent design (where you put tool results), and long conversation management



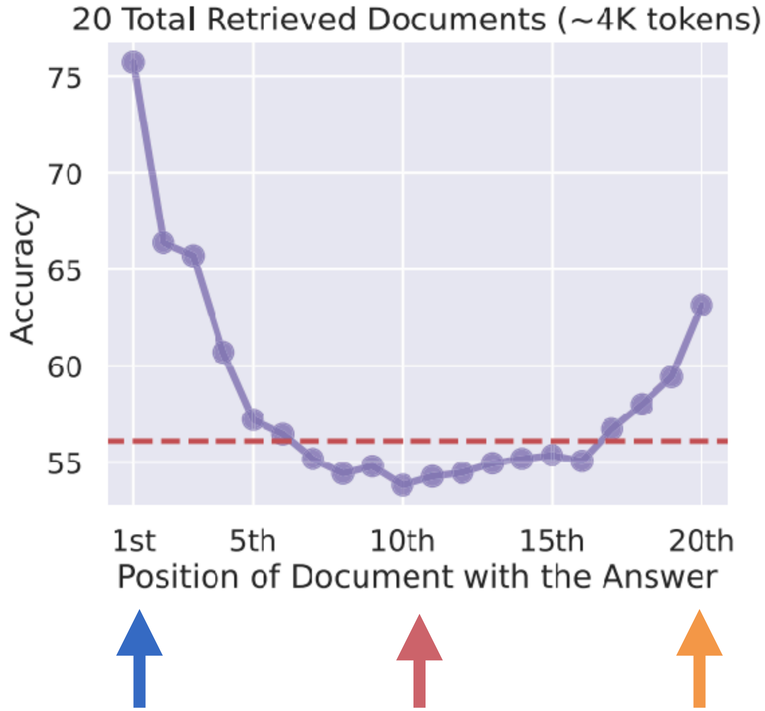
Best when it is at the front (**not bad** at back)

Worst when it is in the middle

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, 12:157–173.

The "lost in the middle" problem

- This is a direct consequence of the self-attention mechanism from last week's reading
  - We know that attention is **distributed across the whole window.**
  - However, that attention **isn't distributed evenly**
    - Every token attends over every other token in the window



Best when it is at the front (**not bad** at back)

Worst when it is in the middle

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics, 12:157–173.

The context window as the engineer's lever

- What you put in the context window determines what the model knows
- What you leave out determines what it has to infer or generate from training
- Managing the context window is the engineer's primary tool for controlling model behaviour
- For an agent running multiple steps, the context window fills up
  - managing what stays, what gets summarised, and what gets dropped is an active engineering problem

The context window as the engineer's lever

How to actually pick?

- We now know the context window as an lever that the engineer controls
- **Our next question:** once the model has produced its probability distribution over the next token, how does it actually pick from it?
  - Maybe always the top token?

- **Temperature controls** the shape of that sampling

Temperature and top-p: controlling randomness

- The model does not always pick the single most probable token — it samples from a distribution
- **Temperature** controls the shape of that distribution:
  - Temperature = 0: always pick the most probable token — deterministic, consistent, predictable
  - Temperature = 1: sample proportionally — creative, varied, sometimes surprising
  - Temperature > 1: flatten the distribution further — increasingly random and incoherent

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. "Distilling the knowledge in a neural network." *arXiv preprint arXiv:1503.02531* (2015).

Top-p: an alternative to temperature

- Temperature reshapes the whole distribution — it doesn't remove any tokens, it just makes the curve ***sharper*** or ***flatter***.
- A different approach: Top-p (nucleus sampling)
  - instead of reshaping, it cuts off the unreliable tail entirely and samples only from the tokens that plausibly belong in the answer.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. "The curious case of neural text degeneration." *arXiv preprint arXiv:1904.09751* (2019).

Top-p: an alternative to temperature

- Top-p (nucleus sampling) samples only from the top tokens whose cumulative probability exceeds p
- Top-p 0.9 means: consider only the tokens that together account for 90% of the probability mass
  - In practice: you rarely need to tune both temperature and top-p, **just pick one**
- Most production use cases: temperature 0 for reliability, temperature 0.7–1.0 for generation

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. "The curious case of neural text degeneration." *arXiv preprint arXiv:1904.09751* (2019).

Switching our focus to “the message”

How a real conversation is actually assembled

- So far today has been about what happens inside a single prediction — tokens, context window, sampling
  - The next few slides zoom out one level: **how a real conversation is actually assembled and sent to the model in the first place**
  - *Token prediction*, *context window limits*, the *"lost in the middle"* effect, and *sampling*, **all operates on this structure**,
- An engineer needs to know what it looks like before designing around it
- This is also where ***system prompts*** live, we’ll briefly test it in today's lab

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. "The curious case of neural text degeneration." *arXiv preprint arXiv:1904.09751* (2019).

Message structure: what the API actually expects

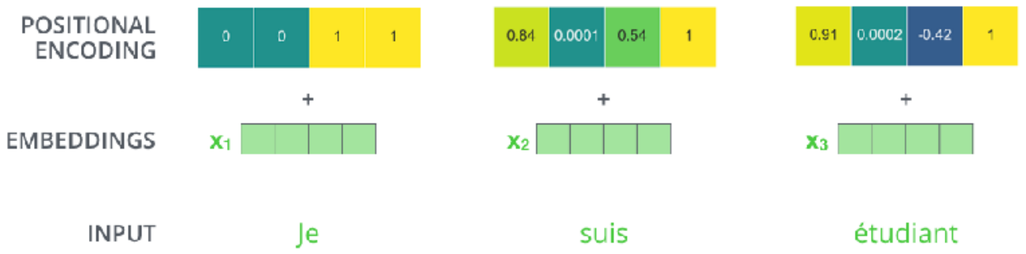
- Three roles: system, user, assistant
  - System prompt: instructions that govern the model's behaviour for the entire conversation — identity, constraints, output format, relevant context
  - User turn: what the human (or calling system) sends
  - Assistant turn: what the model produces — can be prefilled to constrain the response format
- The conversation is a sequence of alternating user and assistant turns, with the system prompt prepended

The system prompt

- **What belongs there**
  - Everything that should govern every response in this context: role definition, output format, constraints, relevant background knowledge
- **What does not belong**
  - Things that change per request (put those in the user turn)
- The system prompt is paid for on every API call — make every token count
- **A good system prompt is specific, not generic**: "You are a helpful assistant" does nothing that “zero-shot” prompting does not already do

Why the order of turns matters

- The model sees the full conversation history on every turn
  - **Recency matters:** the most recent user message has more influence than earlier ones
  - A very long conversation history can dilute earlier instructions — including system prompt instructions
- This is why "prompt injection" attacks work: a sufficiently strong instruction late in the conversation can override earlier constraints



Remember *"Positional Encoding"* from the reading?

Jay Alammar, The Illustrated Transformer, https://jalammar.github.io/illustrated-transformer/

What happens when you send a message, then?

Let’s put it all together

- **You construct a messages array**: system prompt + conversation history + new user message
- This is **tokenized** and sent to the model as a sequence of tokens
- The model **predicts one token at a time**, appending each to the sequence
- It stops when it **predicts a stop token** or **hits the max_tokens limit**
- You receive the generated tokens, decode them back to text, and ***that is the response***

What this means for how you work with models

- You are not issuing commands to an intelligent agent
  — you are ***shaping a probability distribution***
  - The model will produce the most probable continuation of your input
  - Your job as an engineer is to construct inputs that make the right outputs the most probable ones

What we talk about here explain about *prompt engineering*

- **Prompt sensitivity:** small changes shift the probability distribution — different wording, different output
- **Why examples help:** examples constrain what tokens are probable in the response
- **Why "*chain-of-thought*" works:** producing reasoning tokens makes correct answer tokens more probable
- **Why position matters:** tokens near the end of the context influence the next prediction more than tokens in the middle

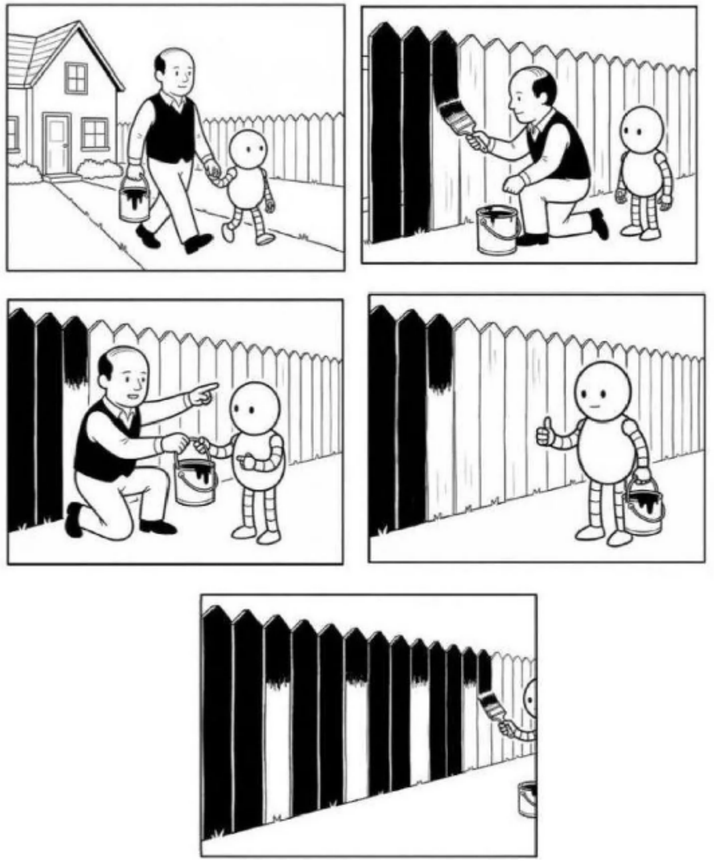
What we talk about here explain about *agents*

- An agent is just repeated token prediction with tool calls in between
  - The agent "thinks" by producing reasoning tokens; it "acts" by producing a structured tool call
  - Context accumulates across steps — the agent's working memory fills up
  - A long-running agent drifts from its original instructions as the context window fills

Common misconceptions

Let’s talk about some things that confuse people

- *“The model knows things”*
  - It has statistical patterns from training data, not knowledge in the human sense
- *"The model understands me”*
  - It predicts probable continuations; understanding is a useful metaphor but not what is happening



Common misconceptions

Let’s talk about some things that confuse people

- *"Temperature makes it more creative”*
  - it makes it less deterministic; the output may be more varied without being more creative in any meaningful sense
- *"A longer context window means better performance”*
  - more context costs more and can hurt performance if it buries the relevant information

Beyond today's lecture

- **Week 5:** when you learn few-shot prompting, you will understand why examples help — they shift the probability distribution
- **Week 6:** when you learn about hallucination, you will understand why it is not a bug — it is prediction under uncertainty
- **Week 10:** when you design a RAG pipeline, you will understand why the context window is the central constraint
- **Week 14:** when you build an agent, you will understand why it drifts — and how to mitigate it

Self-paced module for this week

- **Topic:** in-context learning and instruction tuning
  - Covers: how models learn from examples without updating weights, why raw pre-trained models do not follow instructions, FLAN, InstructGPT, Constitutional AI
  - This week's reading explains **why the same architecture can both be trained on internet text and then follow complex instructions**
- ~60 minutes, complete before next Monday

Today's lab: API exploration

- Three tasks, **individual** lab
- **Task 1: token counting and cost estimation** — five fixed inputs plus your own Mandarin translation, using an offline tokenizer, no account needed
- **Task 2: system prompt sensitivity** — same instruction, three different phrasings, tested in three separate chat sessions,observe how much output shifts on wording alone
- **Task 3: instruction dilution** — set a strict rule at the start of a chat, then keep talking and find out when (or if) it breaks

Today's lab: API exploration

- **Setup note:** none of the primary paths need an API key or account —
  - **Task 1:** `pip install tiktoken`
  - **Tasks 2 and 3:** use a agent that you normally use — ChatGPT, Claude, or Gemini, etc.
    - Optional appendices exist in the handout for anyone with working, funded API access who wants to see it done exactly as described in lecture
    - I will demo the API-written version later in the lab

Lab Task 1: token counting and cost estimation

- Use tiktoken (a real, offline tokenizer — pip install tiktoken, no key, no signup) to measure tokens for:
  - A short English sentence (~15 words)
  - A 200-word English paragraph
  - The same paragraph in Mandarin
  - A Python function (~20 lines)
  - A JSON object with five nested keys
- Print chars, tokens, and chars/token ratio for each — the ratio is what makes the comparison land
- Calculate cost using a real provider's published per-token rate applied to your count
- **Answer:** which surprised you most, and what does it imply? Also: would you expect Claude's or Gemini's real tokenizer to give a higher, lower, or similar number — and why?

Lab Task 2: system prompt grounding

- Same instruction, expressed **three different ways**, tested in three separate chats (on a AI model that you normally would use)
  - **Direct:** "Only use Python's standard library. Never import third-party packages."
  - **Softened:** "You should generally try to stick to the standard library where possible."
  - **Role-framed:** "You're operating in a locked-down environment — only the standard library is installed."
- **Same request each time:** write a function that fetches a URL
- Check the import statement each response reaches for — standard library, or something like requests?
- **Answer:** where does information that must govern every response belong, and what does the gap between outputs tell you about writing your own system prompts?

Lab Task 3: instruction dilution

Push until it breaks

- Open one chat (in any AI model). First message sets a strict rule: respond only in bullet points, no exceptions
- Then just talk — mix neutral chat with narrative/explanatory questions that naturally pull toward prose
- Log approximately where compliance first slips, what kind of question triggered it, and whether it recovers or stays broken
- **Answer:** what would this mean for a long-running agent given a strict instruction at the start of its task?

What to look for across all three tasks

- Notice where your intuitions were wrong
- Notice where a behaviour that surprised you has an explanation in today's lecture
- Add observations to your prompt engineering log — specifically things you did not expect

Before you leave: three questions to sit with

- If the model is predicting tokens, why does it sometimes seem to understand my intent better than I expressed it?
- If the context window is the model's working memory, what happens to an agent that has been running for a long time?
- If temperature 0 is deterministic, why does the same prompt sometimes produce slightly different outputs even at temperature 0?

What to do before next Monday

- Complete the self-paced module: in-context learning and instruction tuning (~60 min)
- Add at least two observations from today's lab to your prompt engineering log
- Form your project team if you have not — team charter due at the start of Week 3's lab

Week 3 preview

- **Topic:** *working with AI in a team*
  - **First half:** team norms, AI-augmented PR, evaluation methodology — the human layer before any building happens
  - **Second half:** the lab — repository setup, draft guidelines, draft evaluation plan
- Come with your team formed and your repository created before class — instructions on the portal

Learning Objectives

✅ Elaborate the relationship between next-token prediction and the range of model behaviors it produces, and why chain-of-thought or examples work at all — as manifestations of one underlying mechanism, not separate quirks

✅ Recognize how an engineer's choices about context window design, message structure, and system prompt wording directly shape how reliably and predictably the resulting system behaves

✅ Evaluate, across your experiments this week — tokenization cost, system prompt compliance, long-conversation drift — where the model's actual behavior matched what you expected going in, and where it didn't

What to do before next Monday

- □ Complete the **self-paced module**: history of language models (~60 min)
  - This will be uploaded today on **the course website (and E3)**.
  - ***Modules are graded from this week***
- □ Submit materials from **today's lab**
  - ***Labs are also graded starting from this week***
- □ **Start to form your project team** — teams of 4 due before Week 3

See you next week!

- Week 3: Working with AI in a team
  - Monday Sep 21, same room, same time
  - Self-paced module due before class
- **Highly suggested:** *Team formed before you arrive*

CSIC30216
AI-in-the-Loop
in Software Project Cycle
AI 導入軟體專案流程

Week 2: How LLMs actually work

Instructors:
Dennis Wang 王邦任
Yu-Chun (Grace) Yen 顏羽君

Date: 2026-09-14